

Quantum Algorithms: QBNTA and QRNTA

M.Chowdhury

Written 24th December 2023*

Revised 15th September 2026*

Draft version (Dates of server time stamps)*

September 15, 2026

Abstract

We give quantum algorithms for gathering numerical evidence about the simple normality of real constants that admit efficient (BBP-type) digit-extraction formulae, such as Stoneham constants and their polynomial generalizations. We first present a Grover-rotation estimator (QBNTA) that prepares a uniform superposition over 2^a digit positions, applies a fixed number of Grover iterations, and estimates the resulting Bernoulli parameter by repeated measurement together with a central-limit-theorem (CLT) confidence test. We show that, as a measurement protocol, this estimator carries the same Fisher information per shot as classical Bernoulli sampling of individual digits obtained directly from the digit-extraction formula, so it achieves no asymptotic speed-up over the natural classical randomized algorithm: both require $O(1/\varepsilon^2)$ samples to resolve a bias of size ε but is still conceptually interesting as a new quantum algorithm. We then show, in an appendix, that replacing repeated measurement with full amplitude estimation (variable-depth Grover iterations combined with quantum phase estimation) does yield a provable quadratic reduction in query complexity, $O(1/\varepsilon)$ versus $O(1/\varepsilon^2)$, and that for the specific null-hypothesis test of interest here (is the limiting proportion of ones equal to $1/2$ or not) only the leading bit of the phase-estimation output is needed to obtain evidence at fixed confidence. We also give our QRNTA algorithm.

1 Introduction

A real number c is *simply normal* in base B if each digit $0, \dots, B-1$ occurs in the base- B expansion of c with limiting frequency $1/B$. Whether well-known constants such as π , e , or $\sqrt{2}$ are simply normal in any base remains open, and empirical evidence for such constants has historically been gathered by computing enormous, contiguous blocks of digits and tabulating frequencies (the classical BNTA, or b -Normal Testing Algorithm, of Bailey et al. see [4]). BBP formulae allow the d -th digit of certain constants, including Stoneham constants

$$\alpha_{b,c} = \sum_{k=1}^{\infty} \frac{1}{c^k b^{c^k}},$$

to be extracted directly, without computing preceding digits, in time polynomial in $\log d$.

The question we address is whether a quantum algorithm exists for a quantum computer can give statistics or empirical result of digit distribution of certain constant. So this question is addressed by using a quantum circuit that implements the digit-extraction structure to test simple normality with fewer oracle calls than a classical algorithm needs. We show that the answer depends delicately on *how* the quantum resource is used: our initial algorithm is for a Grover-rotation-plus-repeated-measurement scheme provides no advantage, while a another version of this algorithm constructed with amplitude-estimation scheme provides a quadratic, advantage. We regard the negative result for the initial scheme as being as important to record for two reasons: because it is conceptually interesting, as the positive result for the refined one, because it is an interesting conceptual contribution - a quantum algorithm need not be faster than some classical algorithm to be conceptually interesting [5]; also because it gives the background motivation for the QAE version of the algorithm which does give a speedup versus the classical comparable algorithm.

Classical BNTA background. For context, we recall the classical b -Normal Testing Algorithm (BNTA), which evaluates digit frequencies directly from the base- B expansion of a constant c . Let

$$g(n, c, d, B) = \begin{cases} 1, & \text{if the } n\text{-th base-}B \text{ digit of } c \text{ equals } d, \\ 0, & \text{otherwise.} \end{cases}$$

BNTA computes the empirical frequency estimator

$$\hat{p}_d(U) = \frac{1}{U} \sum_{n=1}^U g(n, c, d, B),$$

which, when c is simply normal in base B , satisfies

$$\hat{p}_d(U) \longrightarrow \frac{1}{B} \quad \text{as } U \rightarrow \infty, \quad \text{for every } 0 \leq d \leq B - 1.$$

The algorithm must evaluate each of the U terms individually, giving a classical cost of $O(U)$ digit extractions to test a block of U digits. This method (or close variants of it) has been used to gather empirical evidence for the normality or non-normality of specific constants such as π ; whose normality remains a well-known open problem in pure mathematics, and it is the classical baseline against which the quantum algorithms of this paper are compared throughout. In particular, the amplitude-estimation version developed below (AE-QBNTA) achieves a provable quadratic reduction, from $O(1/\varepsilon^2)$ to $O(1/\varepsilon)$ oracle calls, in the number of digit queries needed to resolve a bias of size ε in this estimator ” though, as established in Section 4.3, the naive fixed- K Grover-CLT version does not improve on $\hat{p}_d(U)$ itself, since both rely on the same underlying statistical estimation.

2 Polynomial Stoneham Sums

Generalized Stoneham sums of the form

$$\alpha_{b,c,P} = \sum_{k=1}^{\infty} \frac{1}{c^{P(k)} b^{c^{P(k)}}},$$

where P is a polynomial with positive integer coefficients, retain a polynomial-time digit-extraction algorithm by the same left-half/right-half splitting argument used for $\alpha_{b,c}$ itself: the number of terms needed in the “left half” sum to resolve the n -th base- b digit grows only polynomially in $\log n$, and the “right half” sum converges geometrically. Everything below applies equally to $\alpha_{b,c}$, $\alpha_{b,c,P}$, and sums of finitely many such constants (e.g. the $\alpha_{b,c_1} + \alpha_{b,c_2}$ constants whose b -normality was left open by Bailey and Borwein), provided each summand has such a digit-extraction formula.

3 Generalization to Arbitrary Digit-Extraction Oracles

Everything in this paper is stated for Stoneham-type, polynomial-Stoneham, and Korobov-type constants because these have *polynomial-time* digit extraction, which is what makes the resulting oracle cheap. It is worth being precise, however, that neither QBNTA nor AE-QBNTA actually requires this: both algorithms act on the position register with an oracle that, given n , returns the n -th digit of c , and this oracle can be derived from *any* classical digit-extraction procedure. efficient or not, BBP-type or not, with Bennett’s reversible-computation construction, which converts any classical circuit of size T into a reversible (and hence quantum-usable) circuit of size $O(T)$, up to constant/polynomial overhead. In particular, this applies to the standard BBP formula for π , whose n -th hexadecimal digit is not known to admit a $\text{polylog}(n)$ -time extraction algorithm the way Stoneham constants do: the classical BBP evaluation of this digit requires summing $\Theta(n)$ terms (each reduced with fast modular exponentiation), giving a classical cost on the order of $O(n \log n)$ bit operations, not $O(\text{poly}(\log n))$.

What this means in practice is that QBNTA/AE-QBNTA can be pointed at π , or at any other constant with *some* digit-extraction algorithm, but the number of oracle calls established elsewhere in this paper ” $O(a)$ for fixed- K QBNTA, $O(1/\varepsilon)$ for AE-QBNTA” is multiplied by the classical cost of one such oracle call. For Stoneham constants that per-call cost is polylogarithmic in the digit position, so the overall algorithm remains efficient; for π ’s BBP formula, that per-call cost grows like $\Theta(n \log n)$, so the overall running time inherits this larger cost. We do *not* claim any further quantum speed-up for the internal per-digit computation itself (e.g. reducing this to $O(\sqrt{n})$): the BBP sum for a specific digit must be computed *exactly*, not restricted to to some fixed statistical precision, since a single rounding error near a digit boundary changes the output digit. Quantum amplitude estimation **gives a quadratic speed-up for** *estimating* a sum or proportion to additive precision ε (which is what it is used for elsewhere in this paper, over the position register); it does not, in general, give a quadratic speed-up for *exactly* evaluating a deterministic sum of n terms, a task subject to an $\Omega(n)$ quantum query lower bound in general (the same obstruction that makes exact parity or exact Hamming weight of n bits cost $\Theta(n)$ quantum queries with no Grover-type speed-up). We therefore state the generalization to arbitrary

digit-extraction oracles only at the level Bennett’s construction actually supports: universality of the oracle, not a further speed-up of oracle construction itself. Any genuine reduction in the classical per-digit cost for constants like π would need to come from a separate, different quantum algorithm for that specific summation which is an open question we do not resolve here.

4 The Grover-Central Limit Theorem Algorithm (QBNTA)

Fix a base-2 expansion, a starting position S , and a block length 2^a . Let $f(n) \in \{0, 1\}$ denote the n -th binary digit of c (counted from S), computed with the digit-extraction formula. Prepare the equal superposition

$$\frac{1}{\sqrt{2^a}} \sum_{n=0}^{2^a-1} |n\rangle$$

over the a -qubit position register, and let the oracle mark those n with $f(n) = 1$. Writing M for the number of ones in the block, the initial angle between the “all zero” and “all one” subspaces is

$$\theta = \arcsin\left(\sqrt{\frac{M}{2^a}}\right).$$

Under the hypothesis that c is simply normal, $M \approx 2^a/2$ and $\theta \approx \pi/4$. A single Grover iteration rotates the state by 2θ ; after K iterations the amplitude of the marked subspace is $\sin((2K+1)\theta)$.

How this use of the Grover operator differs from database search.

It is worth being explicit that the way Q is used here is quite different from its use in unstructured search. In ordinary Grover search over N items with a single marked solution, the initial angle $\theta_0 = \arcsin(1/\sqrt{N})$ is small, and one applies a *precomputed* number of iterations $K^* \approx \frac{\pi}{4\theta_0}$ chosen so that the rotated angle $(2K^* + 1)\theta_0$ lands as close as possible to $\pi/2$, the point at which the marked subspace carries essentially all of the amplitude and the unmarked (“bad”) subspace has been driven to near zero. One then measures once, and the bad subspace is not used for anything: it is the thing being eliminated.

QBNTA uses Q differently in two respects. First, it does not stop at a single precomputed K^* chosen to land near $\pi/2$; instead it evaluates the rotated amplitude at a range of values of K , and (in the amplification construction of the previous subsection) K may be taken large enough that $(2K+1)\theta$ sweeps well past $\pi/2$, through π , $3\pi/2$, 2π , and beyond ” winding around the circle multiple times for sufficiently large, still-polynomial K ” rather than being confined to the first quarter-turn. Secondly, QBNTA never tries to drive the “bad” subspace to zero and discard it: both $(\alpha_{0,K})^2$ and $(\alpha_{1,K})^2$ are retained and used as the measured quantities of interest, because it is precisely the *balance* between them ” not the success or failure of isolating one marked outcome ” that carries the statistical information about the digit frequency $M/2^a$. In short, standard Grover search treats the rotation as a means to an end (amplify the marked amplitude, then discard the rest), whereas QBNTA treats the entire trajectory of the rotation, good and bad amplitudes alike, as the object being measured.

4.1 The estimator

QBNTA fixes a small constant K , applies K Grover iterations, measures the register, and records whether the outcome lies in the marked subspace. This is repeated independently $O(a)$ times, giving counts L_0, L_1 with $L_0 + L_1 = O(a)$. The estimator is

$$\hat{p}_K = \frac{L_1}{L_0 + L_1},$$

and under $H_0 : \theta = \pi/4$ (equivalently $M = 2^a/2$) we have $\mathbb{E}[\hat{p}_K] = \sin^2((2K+1)\pi/4) = \frac{1}{2}$.

4.2 Statistical test

Treating each shot as a Bernoulli trial, the CLT gives, under H_0 ,

$$z = \frac{\hat{p}_K - \frac{1}{2}}{\sqrt{0.25/S}}, \quad S = O(a)$$

approximately standard normal, and H_0 is rejected at the usual confidence levels when $|z|$ exceeds the corresponding critical value.

We stress that this normal approximation, and hence the test above, is only valid once S is large enough for the CLT to apply. Asymptotically

$S = O(a)$ shots suffice, as already noted; in practice, the usual rule of thumb for the normal approximation to a Bernoulli sample mean to be reasonable is $S \gtrsim 30$, and this (rather than any quantum consideration) sets the practical floor on the number of repeated measurements below which the z -test above should not be trusted. The same floor applies, for the same reason, to the purely classical random-sampling estimator of Section 4.3, since both use an identical CLT argument on an identical type of Bernoulli data.

4.3 Complexity: no speed-up over classical random sampling

It is not true that the above quantum algorithm, without the refinement below, is comparable to the brute-force type classical BNTA, which computes all 2^a digits in the block explicitly and so costs $O(2^a)$ digit extractions, and to advertise QBNTA's $O(a)$ shots as an exponential improvement. We do not make this claim/comparison, for the following reason.

Each shot of QBNTA, after applying the *fixed*, $O(1)$ -depth Grover operator, yields exactly one classical bit, distributed as $\text{Bernoulli}(\sin^2((2K+1)\theta))$. This is the same type of random variable produced by a classical algorithm that, on each trial, draws a uniformly random position $n \in [0, 2^a)$ and evaluates $f(n)$ directly with the digit-extraction formula (which costs only $\text{poly}(\log(2^a)) = \text{poly}(a)$ time per draw, independent of the block length). Such a classical estimator requires the same $O(1/\varepsilon^2)$ samples, by the same CLT argument, to resolve a bias of size ε in the digit frequency. In other words, the relevant classical baseline for a *statistical* normality test is not “compute every digit in the block” but “draw random digits and estimate their mean,” and against that baseline QBNTA, as described above, offers no asymptotic advantage: both require $O(1/\varepsilon^2)$ queries, and the Fisher information about θ delivered by one QBNTA shot at fixed K is $O(1)$, exactly as for one classical Bernoulli draw. The global, coherent character of the state preparation and the Grover rotation does not change this: by the Holevo bound, a measurement of an a -qubit register yields at most a classical bits of information regardless of how the state was prepared, and in this protocol that information is discarded shot by shot rather than accumulated coherently across shots.

We record this as a feature of the algorithm: it correctly reproduces the classical statistical test, it can be executed on a quantum device, but it is not

by itself evidence of a quantum speed-up for normality testing. A speed-up, when one exists, must come from a different use of the same oracle; we give one such construction in the appendix.

4.4 An optional, non-polynomial amplification

For completeness we note an optional refinement that trades away polynomial-time complexity in exchange for the ability to detect arbitrarily small biases. When the block length 2^a is large, write the Grover angle as $\theta = \frac{\pi}{4} \pm \theta_{\text{error}}$, where $\theta_{\text{error}} \rightarrow 0$ as $a \rightarrow \infty$ under the simple-normality hypothesis. After K Grover iterations the rotation angle becomes $(2K+1)\theta$, so the deviation from the ideal value $\pi/4$ is amplified to $(2K+1)\theta_{\text{error}}$ (reduced mod $\pi/2$). If the user specifies a tolerance π/E and chooses K such that $(2K+1)\theta_{\text{error}} < \pi/E$, the small-angle approximation $\sin((2K+1)\theta_{\text{error}}) \approx (2K+1)\theta_{\text{error}}$ applies, giving

$$\left| \frac{1}{2} - (\alpha_{0,K})^2 \right| \approx (2K+1)\theta_{\text{error}}, \quad \left| \frac{1}{2} - (\alpha_{1,K})^2 \right| \approx (2K+1)\theta_{\text{error}}.$$

Thus, by increasing K beyond the polynomial regime used in QBNTA, one may amplify extremely small deviations from simple normality until they are large enough to resolve by direct (fixed- K) sampling as in Section 4.3. This provides an optional, non-polynomial-time refinement that can strengthen empirical evidence for non-normality or abnormality when the underlying bias is too small to be detected by the polynomial-time version of the algorithm. We emphasize that this amplification is purely optional: it does not preserve polynomial-time complexity, and the algorithm's user need not invoke it.

We note, to avoid over-claiming, that this is not a capability beyond amplitude estimation: resolving a deviation of order θ_{error} this way requires $K = O(1/\theta_{\text{error}})$ Grover iterations, i.e. $O(1/\theta_{\text{error}})$ oracle calls per shot, which is the same order of oracle calls that the amplitude-estimation circuit of Appendix A uses to resolve the same precision directly. Unlike amplitude estimation, a single measurement after K fixed iterations still only yields one Bernoulli sample of the amplified bias, so repeated CLT-style sampling is still required on top of the rotation to resolve it reliably; amplitude estimation reads the phase off in one coherent measurement (up to its stated error bound) without this further sampling step. The two constructions are best understood as a cruder, single-fixed- K special case and generalization, at comparable query cost, rather than as two independent sources of advantage.

5 An Irrationality Criterion

The digit statistics gathered above also bear on the (harder) question of whether c is irrational, with the following criterion.

Theorem 1 (Irrational-limit criterion). *Let c be a real number with a digit-extraction formula (e.g. BBP), and let $c_O(H)$ and $c_Z(H)$ denote, respectively, the number of ones and zeros among the first H binary digits of c . If the limit*

$$l = \lim_{H \rightarrow \infty} \frac{c_O(H)}{H} = 1 - \lim_{H \rightarrow \infty} \frac{c_Z(H)}{H}$$

exists and is irrational, then c is irrational.

Proof sketch. If c is rational, its binary expansion is eventually periodic with some period π_0 . The digit counts $c_O(H)$ and $c_Z(H)$ then satisfy $c_O(H)/H \rightarrow l_0$, where l_0 is the (rational) proportion of ones in one period, since the pre-periodic part contributes a vanishing fraction as $H \rightarrow \infty$. Hence if c is rational, the limit l , when it exists, is rational. Contrapositively, if l exists and is irrational, c cannot be rational. $\square$

Remark 2. Since $l = \frac{1}{2}$ is rational, an irrational value of l automatically satisfies $l \neq \frac{1}{2}$. Consequently the hypothesis of Theorem 1 can only ever be met by a constant that is *not* simply normal (in base 2): the theorem never fires in the simply-normal case $l = 1/2$, and it is silent when l is rational but not equal to $1/2$ (an abnormal constant with a rational bias, which the theorem does not detect). In short, Theorem 1 is a criterion that applies only within the abnormal case, and only to the sub-case of abnormality in which the limiting frequency itself is irrational.

This criterion is one-directional: it gives no information when l is rational (in particular it says nothing when $l = 1/2$, the simply-normal case), and it presupposes that the limit l exists and can be estimated with enough confidence to determine whether it is rational or irrational, which takes more steps to do than the empirical task than the finite-confidence hypothesis tests used elsewhere in this paper. We record it here because it gives the Grover-angle discussion below its intended interpretation.

Connection to the Grover-angle behaviour. The criterion of Theorem 1 has a natural, but limited, connection to the Grover angle

$$\theta_a = \arcsin\left(\sqrt{\frac{M}{2^a}}\right),$$

where M is the number of ones in the length- 2^a block used by QBNTA. If the limiting proportion $l = \lim_{H \rightarrow \infty} c_O(H)/H$ is irrational, then $M/2^a \rightarrow l$ as $a \rightarrow \infty$, so $\theta_a \rightarrow \arcsin(\sqrt{l})$, which is again irrational (as a multiple of π , generically) in the limit. When θ_a/π is irrational, the Grover-rotated angles $(2K+1)\theta_a \bmod 2\pi$ are Weyl-equidistributed as $K \rightarrow \infty$ rather than cycling through a finite set of values, so the amplitudes $(\alpha_{0,K})^2$ and $(\alpha_{1,K})^2$ behave erratically across K matching outcome (v) of QBNTA.

Caveat. At any *finite* block length a , $M/2^a$ is itself a rational number, and by Niven's theorem $\arcsin(\sqrt{x})$ is a rational multiple of π , for rational x , only at $x \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$. For every other value of $M/2^a$ " which is the generic case, including finite blocks drawn from numbers with a perfectly ordinary *rational* bias such as $l = 1/3$ " θ_a is already an irrational multiple of π , and outcome (v) already occurs, with no relation to whether the true asymptotic limit l is rational or irrational. Consequently, observing outcome (v) at a single, fixed block length a is *not* by itself evidence that l is irrational: it is the expected behaviour for almost any bias other than the five Niven-special ratios above. A genuine irrationality signature would require the erratic behaviour to persist, without resolving into a periodic pattern, as the block length $a \rightarrow \infty$ (not restricted to as K is increased at fixed a) and even then, ruling out an ordinary rational bias simply avoids the Niven-special ratios requires an argument beyond the statistics of any one block. We therefore present this connection as an interpretive link between Theorem 1 and outcome (v), not as an independent quantum test for irrationality.

6 Outcomes of the Statistical Test

Independently of the complexity question above, the five qualitative outcomes of the test remain useful for organizing empirical results:

1. Exact balance: $\hat{p}_K = \frac{1}{2}$ for all tested K , consistent with H_0 .
2. All ones observed in the block (a Brouwer-type result with 1s in place of 7s).

3. All zeros observed in the block.
4. $\hat{p}_K$ differs from $\frac{1}{2}$ by a small, consistent amount ε across K , evidence of a detectable digit bias.
5. $\hat{p}_K$ behaves erratically across K , consistent with an irrational θ and "Weyl-equidistributed rotation angles" evidence against simple normality of a different character than case 4.

6.1 QBNTA as a classical-filter subroutine

Because QBNTA is a probabilistic estimator, outcome (i) "the case in which $\hat{p}_K = \frac{1}{2}$ for all tested K " provides statistical evidence, at whatever confidence level the CLT test was run at, that the tested block B has close to equal numbers of zeros and ones. In some applications one may separately want to know *exactly* whether B has an equal count, which a full classical check can settle by evaluating all 2^a digits in B , at cost $\Theta(2^a)$.

QBNTA can serve as a cheap pre-filter ahead of such a check: run QBNTA first; only if the outcome is consistent with exact balance (case (i)) is it worth paying the $\Theta(2^a)$ cost of the exact classical count. If QBNTA instead reports a clear bias or erratic behavior (cases (iv) or (v)) at high confidence, the exact check can usually be skipped, though we note this is a statistical inference, not a certainty, since the CLT test has a nonzero (if small, and controllable with the confidence level) false-negative rate. Run across many candidate blocks, of which only a small fraction are actually near-balanced, this filter-then-verify strategy reduces the *expected* total classical work, since most blocks are dismissed at the $\text{poly}(a)$ filtering stage rather than paying $\Theta(2^a)$ each.

We are careful, however, not to attribute this expected saving to QBNTA being a *quantum* algorithm. As established in Section 4.3, the fixed- K , repeated-measurement version of QBNTA has exactly the statistical power of classical random sampling of digits at the same $\text{poly}(a)$ cost, so an ordinary classical statistical pre-filter, built the same way, would serve exactly as well as a front end to the $\Theta(2^a)$ exact check, with no quantum computer involved at all. The filtering benefit here is a property of the filter-then-verify *strategy* (cheap necessary-condition test before an expensive exact check), not a consequence of using Grover rotations. But a quantum-attributable speedup does enter is if AE-QBNTA (Appendix A) is used as the filter in place

of fixed- K QBNTA: this reduces the query cost of the filtering stage itself by the same quadratic factor established there, $O(1/\varepsilon)$ versus $O(1/\varepsilon^2)$ oracle calls to reach a given confidence. This is a real, if modest, improvement to the filter’s own cost ” it is a quadratic saving in the filtering step, not an exponential one, and it does not change the $\Theta(2^a)$ worst-case cost of the exact verification step whenever that step is actually invoked.

7 A Quantum Algorithm for Testing Rationality with Digit Oracles

In this section we describe a quantum algorithm (QRNTA) that uses the same digit-extraction oracles developed earlier for Stoneham, polynomial Stoneham, and Korobov-type constants to test whether a real number c is rational in a given base b . A number is rational in base b if and only if its base- b expansion is eventually periodic. QRNTA is a quantum analogue of the classical period-finding procedure used in Shor’s algorithm, using the BBP oracle for digit extraction together with the quantum Fourier transform (QFT) to detect periodicity in the digit sequence.

Theorem 3 (Quantum Rationality Testing Algorithm). *Let c be a real number with a polynomial-time digit-extraction formula (e.g. a BBP-type formula). Then there exists a quantum algorithm (QRNTA) that, for any chosen base b , gives numerical evidence for whether the base- b expansion of c is eventually periodic. If a period is detected, the algorithm outputs a candidate period e and verifies it classically. If no period is detected, the algorithm provides evidence that c is irrational in base b .*

Proof

Step 1: Classical pre-test. Choose a base b and a starting position d . Using the BBP digit oracle, compute the values

$$b^d c \bmod 1, \quad b^{d+1} c \bmod 1, \quad \dots, \quad b^{d+a} c \bmod 1$$

for a polynomial number of consecutive positions. If several consecutive values are close to integers, this suggests that the expansion may have terminated or entered a short period. Otherwise, proceed to the quantum period-finding stage. (In what follows, d is assumed large enough that c ’s expansion,

if periodic at all, is already in its periodic part from position d onward; this is the same assumption implicit in the classical pre-test.)

Step 2: Construct a 1-1 Oracle for the Period. Fix a window length L , polynomial in the number of qubits used, and at least as large as the largest candidate period the algorithm is intended to certify (see the remark below). Define the digit-window encoding

$$P_L(n) = \sum_{i=0}^{L-1} (\delta_{\varkappa_i}(L)) b^i R(n+i, c, b)$$

where

$$\begin{aligned} \delta_{\varkappa_i}(L) &= 1 \text{ if } L = \varkappa_i \\ \delta_{\varkappa_i}(L) &= 0 \text{ if } L \neq \varkappa_i \end{aligned}$$

$\delta_{\varkappa_i}(L)$ is an indicator function where $\varkappa_1, \varkappa_2, \dots$ is a sequence of number where the sequence contains two or more zeroes (that are not the first or the last term). L is a function that grown polynomially in $O(\log(n))$ e.g. $L = 3\text{floor}(\log(n)) + 1$. i.e. the L polynomial spaces, need not be necessarily consecutive as the above formula for L expresses for example', as a function of base- b digits of c starting at position n , read as a single base- b integer of L digits, where $R(n, c, b)$ denotes the n -th base- b digit of c . This map is computable in polynomial time with L calls to the digit-extraction oracle, and it is manifestly injective on windows: two starting positions give the same value of P_L if and only if the L -digit windows starting there are identical. Prepare a quantum register $|n\rangle|0\rangle$ and apply the oracle

$$|n, 0\rangle \longrightarrow |n, P_L(n)\rangle.$$

Remark (why L must dominate the candidate period). If the base- b expansion of c is periodic with period e from position d onward, then $R(n, c, b) = R(n+e, c, b)$ for all $n \geq d$, and hence $P_L(n) = P_L(n+e)$ for all $n \geq d$: the digit-window function inherits the period e . For the QFT stage below to see a clean arithmetic progression of positions sharing one encoded value " rather than spurious coincidences between unrelated phases " the window must be long enough to distinguish the e distinct phases within one period, which requires $L \geq e$. This is the same requirement, expressed in different analogous language, that the second register in Shor's order-finding

algorithm be large enough to make the modular exponentiation function injective on one period.

An alternative, more compact digit-window encoding (conjectural).

The encoding P_L above is unconditionally injective once $L \geq e$, but it pays for that guarantee with a window of size $L = \Theta(e)$, which may be prohibitively large if one wants QRNTA to be able to certify periods e that are themselves large (e.g. exponential in the size of the position register). We record here an alternative encoding, adapted from the *Binary Zig-Zag construction* of an unpublished companion paper on Pisano periods [3], which is conjectured "not proved" to achieve injectivity on one period using a window of size only $O(\log e)$ rather than $\Theta(e)$. We present it as an optional alternative to P_L , to be used only if one is willing to accept its conjectural status in exchange for the shorter window; every other part of QRNTA (Steps 3–7) is unaffected by which encoding is used for Step 2.

Reduce each digit to a single bit, $p(n) = R(n, c, b) \bmod 2 \in \{0, 1\}$, and fix a pseudo-random generator built from Weyl's equidistribution theorem,

$$\text{PRNG}(\lambda, k) = (k\lambda) \bmod 1, \quad (701)$$

for a fixed irrational λ (e.g. $\lambda = \sqrt{p} \bmod 1$ for a prime p , computable in polynomial time); since λ is irrational, $(\text{PRNG}(\lambda, k))_{k \geq 0}$ is equidistributed mod 1 by Weyl's criterion, unconditionally.

Fix bound parameters η, μ and a number of layers ϖ , all polynomial in n (empirically, $\varpi = O(\log e)$ is what one conjectures then will suffice, by analogy with Conjecture 2 of [3], though as in that paper this growth rate is an empirical observation, not a derived bound). For each layer $\sigma = 1, \dots, \varpi$, fixed known irrationals ξ_σ, τ_σ (e.g. square roots of the $(2\sigma + 1)$ -th and 2σ -th primes, respectively, reduced mod 1), and window position $i = 0, \dots, w - 1$, define the *period layer function*

$$\text{PL}_\sigma(n+i) = \left(p(n+i) \cdot \left\lfloor \eta \text{PRNG}(\xi_\sigma, p(n+i)) \right\rfloor + \left\lfloor \mu \text{PRNG}(\tau_\sigma, p(n+i)) \right\rfloor \right) \bmod 2.$$

Since PL_σ is a fixed deterministic function applied to $p(n+i)$, and $p(n+i)$ is periodic in n with period dividing e (inherited from $R(\cdot, c, b)$), $p(n+i)$ is equal to $R(\cdot, c, b)$ here, to $\text{PL}_\sigma(n+i)$ is *unconditionally* periodic in n with period dividing e as well this part requires no conjecture, exactly as in the period-compatibility lemma of [3].

(Note we can replace λ by a fixed BBP formula in 701, for $\ln 2$,

$$\xi_\sigma = V(\sigma) \ln 2 = \sum_{j=1}^{\infty} \frac{V(\sigma)}{j2^j}$$

$$\tau_\sigma = V'(\sigma) \ln 2 = \sum_{j=1}^{\infty} \frac{V'(\sigma)}{j2^j}$$

, $V(\cdot)$ and $V'(\cdot)$ are integer positive polynomial in σ similar for τ_σ etc, optionally since $\text{PRNG}(\lambda, k)$ is evaluated at a polynomial number of times above.). $V(\cdot)$ and $V'(\cdot)$ can be $\text{Floor}(3\sigma+1+abs \log[abs(10\sigma \log(R(n, c, b)))])$ and $\text{Floor}(\sigma^2+2+abs \log[abs(20.5\sigma \log(R(n, c, b)))])$ respectively for example. Another alternative for $V(\cdot)$ and $V'(\cdot)$ of the form $O(2^\sigma)$ and $O(2^{\sigma^2})$ using binary exponentiation but that assumes the normlarity of $\ln 2$ unlike our other construction above which is based on the Weyl Criterion.

What is *not* unconditional is whether interleaving ϖ such layers across a window of w positions,

$$Q_{w,\varpi}(n) = \sum_{i=0}^{w-1} \sum_{\sigma=1}^{\varpi} 2^{i\varpi+\sigma-1} \text{PL}_\sigma(n+i),$$

recovers the full period e exactly (equivalently, is injective on one period), rather than collapsing onto some proper divisor of e if the layers happen to collide. We state this explicitly as its own conjecture, distinct from (though motivated by the same heuristic as) the Binary Zig-Zag Injectivity Conjecture of [3]: that conjecture concerns Fibonacci residues, which have an algebraic recurrence structure that this construction's digit sequence $R(\cdot, c, b)$ need not share, so its heuristic support does not automatically transfer here.

Conjecture 4 (Digit Zig-Zag Injectivity Conjecture, adapted from [3]). *There exists a constant C such that, for $w\varpi \geq C \log_2 e$, the map $n \mapsto Q_{w,\varpi}(n)$ is injective as a function of $n \bmod e$, for a suitable choice of η , μ , and the irrationals ξ_σ, τ_σ .*

Remark 5 (Heuristic support for the conjecture). We have no proof of this conjecture, but the same style of probabilistic heuristic that [3] offers in support of its own Binary Zig-Zag Injectivity Conjecture applies here, and in one respect applies in a stronger form, because of the extra freedom this construction has in choosing the layer parameters.

Model each single-bit layer output $PL_\sigma(n+i)$, for a *generic* (not adversarially chosen) choice of η, μ and ξ_σ, τ_σ , as "behaving like an independent, unbiased Bernoulli(1/2) trial" precisely the modeling assumption [3] uses for its own single-bit projection P_n . Under this model, $Q_{w,\varpi}(n) \bmod 2^{w\varpi}$ carries $w\varpi$ heuristically-independent bits. Two distinct positions $n \neq n'$ within one period then agree on all $w\varpi$ bits with probability $\approx 2^{-w\varpi}$, and a union bound over the $\binom{e}{2}$ pairs of positions in one period gives an expected number of colliding pairs of about $\binom{e}{2}2^{-w\varpi}$, which drops below 1 once $w\varpi \gtrsim 2 \log_2 e$ "reproducing, with the identical argument, the same $O(\log e)$ threshold asserted above and in Conjecture 3 of [3].

There is a further point specific to this construction that strengthens the heuristic, though it still falls short of a proof. The conjecture only requires *some* choice of $\eta, \mu, \xi_\sigma, \tau_\sigma$ to work, not every choice " and the construction has substantial freedom here: $\varpi = O(\log e)$ layers, each with its own pair of irrationals (the primes-based choice given above is one canonical example, not the only option), and bound parameters η, μ free to range over any polynomial-sized set. Varying these choices gives access to a large number of qualitatively different realizations of $Q_{w,\varpi}$ " how large, precisely, we do not derive here, beyond noting that it grows with e rather than staying fixed, in the same spirit as (though we do not commit to a specific rate matching) the empirically-fit growth constants [3] reports for its own conjecture. If different realizations behave close to independently of one another, then even if some particular, unlucky choice of parameters collapses the period (so that a proper divisor of e appears instead of e itself), the availability of many alternative, roughly-independent realizations makes it plausible that at least one avoids this failure " the same "probabilistic method" shape of argument used to show combinatorial objects exist without exhibiting one directly.

We flag the same limitation [3] flags for its own analogous heuristic: this argument treats layer outputs, and different parameter choices, as if they were genuinely random, when both are in fact deterministic functions of a common underlying digit sequence and of each other. True independence " across positions within a period, or across different choices of $(\eta, \mu, \xi_\sigma, \tau_\sigma)$ " is exactly what is *not* established here. The heuristic is offered as evidence that the conjecture is plausible and worth recording, not as a proof, and it would require converting the "many available choices" observation above into a rigorous existence argument without further work.

We regard this only as a plausible, untested extension of an already-open

conjecture from an unpublished source, offered as an optional route to a more compact oracle; QRNTA's correctness as stated in Steps 1–7 relies only on the unconditional P_L encoding of Step 2, and does not require this conjecture to hold.

Step 3: Prepare a superposition over candidate positions. Prepare the state

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |P_L(a)\rangle \text{ (or alternatively } \frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |Q_{w,\varpi}(a)\rangle)$$

Measuring the second register yields a value k , collapsing the first register to a uniform superposition over all positions a whose digit window encodes to k . If the expansion is periodic with period e (and $L \geq e$, as above), these positions form an arithmetic progression

$$A = \{u, u + e, u + 2e, \dots, u + (M - 1)e\},$$

so the post-measurement state is

$$\frac{1}{\sqrt{M}} \sum_{w=0}^{M-1} |u + we\rangle |k\rangle.$$

Step 4: Apply the quantum Fourier transform. Applying the QFT to the first register yields

$$\frac{1}{\sqrt{qM}} \sum_{v=0}^{q-1} \left(\sum_{w=0}^{M-1} e^{2\pi i(u+we)v/q} \right) |v\rangle |k\rangle.$$

The inner sum is a geometric series in the root of unity $\Omega = e^{2\pi iev/q}$.

Step 5: Measure the Fourier register. The probability of observing v is

$$\Pr(v) = \frac{1}{Mq} \left| \sum_{w=0}^{M-1} \Omega^w \right|^2.$$

If ev/q is close to an integer, then $\Omega \approx 1$ and the geometric sum has magnitude $\approx M$, giving $\Pr(v) \approx 1/q$. Otherwise the sum nearly cancels and $\Pr(v)$ is small. Thus the observed values v satisfy

$$\frac{ev}{q} \approx L'$$

for some integer L' (renaming the integer here to avoid clashing with the window length L of Step 2).

Step 6: Recover the period using continued fractions. Given the observed value v , compute the convergents of v/q with continued fractions. If $v/q \approx L'/e$, then the denominator of a convergent provides a candidate period e . This step is classical and polynomial-time.

Step 7: Classical verification. Verify the candidate period e by checking whether

$$b^d c (b^e - 1)$$

is an integer, using the BBP formula for c . (This is the standard periodicity identity: if c 's base- b expansion is periodic with period e from position d onward, then $z = \text{frac}(b^d c)$ is purely periodic with period e , so $z(b^e - 1) \in \mathbb{Z}$; since $\lfloor b^d c \rfloor (b^e - 1)$ is trivially an integer, their sum $b^d c (b^e - 1)$ is an integer as well.) If it is an integer, the expansion is periodic with period e , and c is rational in base b . If no such e is found, the measurements appear random, providing evidence that c is irrational in base b .

Discussion. QRNTA is a quantum analogue of classical rationality testing with period detection, but it replaces exponential classical search with a polynomial-time BBP oracle and a QFT-based period-finding procedure. In this section period-finding with the QFT is a task of the kind for which a speed up (here, exponential-in- $\log e$) quantum advantage over the classical search was established. The algorithm applies directly to Stoneham constants, polynomial Stoneham constants, finite Stoneham sums, and Korobov-type constants, since all of these admit polynomial-time digit-extraction oracles. Thus QRNTA provides a unified quantum framework for gathering numerical evidence of rationality or irrationality across a broad class of BBP-type constants.

8 Conclusion

We have described QBNTA, a quantum protocol for statistically testing simple normality of constants with efficient digit-extraction formulae. Used with a fixed number of Grover iterations and repeated measurement, QBNTA reproduces a valid classical statistical test but provides no asymptotic speed-up over classical random sampling of digits, since both approaches extract the same $O(1)$ Fisher information per query and require $O(1/\varepsilon^2)$ queries overall. A genuine, though only quadratic, quantum speed-up is available by replacing repeated measurement with full amplitude estimation; we develop this in the appendix, together with a simplification specific to the null-hypothesis test considered here.

References

- [1] Locate ref
- [2] Wall, Locate ref.
- [3] A Quantum Algorithm for Zig-Zag Functions and a Grover-Based Algorithm for the Golden Mean, Unpublished manuscript on quantum period-finding for Pisano periods with a “Zig-Zag” interleaving construction, introducing the Zig-Zag and Binary Zig-Zag Injectivity Conjectures
- [4] LOcate ref, Classival BNTA algorithm
- [5] Locate ref
- [6] Locateref

A Amplitude Estimation Version and Its Quadratic Speed-up

We now replace the fixed- K , repeated-measurement protocol of Section 4.3 with the standard quantum amplitude estimation (QAE) construction of Brassard, Høyer, Mosca, and Tapp, applied to the same oracle.

A.1 Construction

Let A denote the state-preparation operator producing the uniform superposition $\sum_n |n\rangle/\sqrt{2^a}$, and let Q denote the Grover operator built from A and the digit oracle, as before. Instead of applying Q a fixed number of times and measuring, QAE prepares an ancillary register of t qubits in equal superposition and applies Q^{2^j} controlled on the j -th ancilla qubit, for $j = 0, \dots, t-1$, followed by an inverse quantum Fourier transform on the ancilla register. This is exactly phase estimation applied to Q , whose eigenvalues are $e^{\pm 2i\theta}$. Measuring the ancilla register yields an integer y from which an estimate

$$\hat{\theta} = \pi y / 2^t$$

is recovered, satisfying, with probability at least $8/\pi^2$,

$$|\hat{\theta} - \theta| \leq \frac{\pi}{2^t}.$$

A.2 Query complexity

Each controlled- Q^{2^j} costs 2^j oracle calls, so a single run of this circuit costs $O(2^t)$ oracle calls and yields an estimate of θ (equivalently of $M/2^a$) accurate to $O(2^{-t})$. Setting $2^{-t} = \varepsilon$ gives an estimate of θ to precision ε using $O(1/\varepsilon)$ oracle calls. This is the standard quadratic improvement over the $O(1/\varepsilon^2)$ oracle calls required by direct sampling (Section 4.3), and it is provable: it follows directly from the BHMT amplitude estimation error bound and does not depend on any unproven assumption about c . It is also, we stress, only quadratic; it is not exponential, and no construction of this kind can be, since a measurement of $t = O(\log(1/\varepsilon))$ ancilla qubits cannot carry more than $O(\log(1/\varepsilon))$ bits of information about a real parameter without further structure, and the $O(1/\varepsilon)$ query count is itself already known to be information-theoretically optimal for this task (amplitude estimation matches the known lower bound for the corresponding query model).

A.3 Simplification for the balanced/unbalanced null-hypothesis test

For the specific test used in the body of this paper, we do not need a fully precise estimate of θ ; we only need to decide between $H_0 : \theta = \pi/4$ and a fixed-size deviation from it. In the amplitude estimation output $\hat{\theta} = \pi y / 2^t$,

the hypothesis $\theta = \pi/4$ corresponds to $y = 2^{t-2}$, i.e. to the leading bit of y (in an appropriate normalization) being fixed and the remaining bits vanishing. Consequently, for a null-hypothesis test at fixed confidence, it suffices to read out the leading bit of the phase-estimation register: if that bit disagrees with the value predicted under H_0 , this already constitutes evidence, at the corresponding confidence level, that $\theta \neq \pi/4$ and hence that the digit frequency is biased away from $\frac{1}{2}$. This reduces the constant factor (and, for a one-bit test at fixed confidence, the required value of t) needed in practice, without changing the $O(1/\varepsilon)$ asymptotic query complexity of the underlying amplitude-estimation circuit; it is a statement about how much of the phase-estimation output must be trusted for this particular binary decision, not a further asymptotic speed-up beyond the quadratic one already established above.

A.4 Summary

- **Fixed- K , repeated measurement, CLT (main text):** no speed-up over classical BBP-based random sampling of digits; both require $O(1/\varepsilon^2)$ queries; the Fisher information per query is $O(1)$ in both cases.
- **Variable-depth Grover + quantum phase estimation (this appendix):** a provable quadratic speed-up, $O(1/\varepsilon)$ oracle calls versus $O(1/\varepsilon^2)$; for the specific null-hypothesis test of interest, only the leading bit of the phase-estimation output is needed to reach a decision at fixed confidence. This is not an exponential speed-up, and no construction along these lines can be, by the same information-theoretic limits noted above.

B Full QBNTA, Generalized QBNTA, and Stoneham Oracles

B.1 B.1 Full QBNTA (Quantum B-Normal Testing Algorithm)

Algorithm 1 QBNTA: Quantum B-Normal Testing Algorithm (Base 2)

[1]

Input:

- A real constant c with a BBP-type digit-extraction formula.
- Starting position S and ending position P with $P - S = 2^a$.
- Oracle $f(n, c, S, P)$ returning the $(n + S)$ -th binary digit of c .
- Number of Grover iterations K (fixed constant).

Initialize:

- Prepare the uniform superposition:

$$|I\rangle = \frac{1}{\sqrt{2^a}} \sum_{n=0}^{2^a-1} |n\rangle.$$

- Counters $L_0 = 0, L_1 = 0$.

Oracle marking:

$$U_f|n\rangle|w\rangle = |n\rangle|w \oplus f(n, c, S, P)\rangle.$$

Grover iteration: Apply the Grover operator $Q = -AS_0A^{-1}S_f$ exactly K times.

Measurement: Measure the position register. If $f(n, c, S, P) = 1$ increment L_1 , else increment L_0 .

Repeat: Repeat Steps 3–5 independently $O(a)$ times.

Estimator:

$$\hat{p}_K = \frac{L_1}{L_0 + L_1}.$$

CLT Test: Under $H_0 : M = 2^a/2$,

$$z = \frac{\hat{p}_K - 1/2}{\sqrt{0.25/(L_0 + L_1)}}.$$

Output: One of the five outcomes:

1. $(\alpha_{0,K})^2 = (\alpha_{1,K})^2 = 1/2$ (exact balance).
2. $(\alpha_{0,K})^2 = 0, (\alpha_{1,K})^2 = 1$ (all ones).
3. $(\alpha_{0,K})^2 = 1, (\alpha_{1,K})^2 = 0$ (all zeros).
4. $(\alpha_{0,K})^2, (\alpha_{1,K})^2 \approx 1/2$ but not equal (small bias).
5. $(\alpha_{0,K})^2, (\alpha_{1,K})^2$ erratic (Weyl-distributed).

B.2 B.2 Generalized QBNTA for Base B

Algorithm 2 Generalized QBNTA (Base B)

[1]

Input:

- A real constant c with a BBP-type digit-extraction formula in base B .
- Starting position S and ending position P with $P - S = 2^a$.
- Oracle $f_B(n, c, S, P)$ returning the $(n + S)$ -th base- B digit of c .
- Target digit $d \in \{0, \dots, B - 1\}$ whose frequency is being tested.
- Number of Grover iterations K (fixed constant).

Initialize:

- Prepare the uniform superposition:

$$|I\rangle = \frac{1}{\sqrt{2^a}} \sum_{n=0}^{2^a-1} |n\rangle.$$

- Counters $L_d = 0$, $L_{\neg d} = 0$.

Digit oracle:

$$f_B(n, c, S, P) = \text{IntegerPart}(B \cdot \text{Frac}(B^{n+S} \cdot \text{BBP}(c, B))).$$

Oracle marking:

$$U_{f_B, d} |n\rangle |w\rangle = |n\rangle |w \oplus \mathbf{1}[f_B(n, c, S, P) = d]\rangle,$$

so that positions with $f_B(n, c, S, P) = d$ are marked, and all other digit values $0, \dots, B - 1 \setminus \{d\}$ are treated as a single unmarked class – the same binary good/bad partition used in Algorithm 1, now built around the fixed target digit d rather than the binary value 1.

Grover iteration: Apply the Grover operator $Q_d = -AS_0A^{-1}S_{f_B, d}$ exactly K times, where $S_{f_B, d}$ reflects about the marked subspace $\{n : f_B(n, c, S, P) = d\}$.

Amplitude geometry: If digit d appears with frequency $1/B$, then

$$\theta_d = \arcsin \left(\sqrt{\frac{1}{B}} \right).$$

Measurement: Measure the position register. If $f_B(n, c, S, P) = d$ increment L_d , else increment $L_{\neg d}$.

Repeat: Repeat Steps 6–7 independently $O(a)$ times.

Estimator:

$$\hat{p}_{K,d} = \frac{L_d}{L_d + L_{\neg d}}.$$

CLT test: Under $H_0 : \Pr[f_B = d] = 1/B$,

$$z_d = \frac{\hat{p}_{K,d} - 1/B}{\sqrt{(1/B)(1 - 1/B)/(L_d + L_{\neg d})}}.$$

Output: Generalized versions of outcomes (i)–(v) from Algorithm 1, now stated for the fixed target digit d .

Remark 6. Algorithm 2 tests the frequency of one fixed target digit d per run, exactly mirroring Algorithm 1’s binary marked/unmarked partition (there, implicitly, $d = 1$). To gather evidence about all B digit frequencies of c , this whole procedure is repeated once for each $d \in \{0, \dots, B - 1\}$ (or for $B - 1$ of them, since the remaining frequency is then determined by the constraint that all B frequencies sum to 1); it is not obtained from a single run’s measurements, since a single Grover reflection is defined relative to one marked subspace at a time.

B.3 B.3 Oracle for a Single Stoneham Constant

$$\alpha_{b,c} = \sum_{j=1}^{\infty} \frac{1}{c^j b^{c^j}}.$$

Digit oracle:

$$f(n, c, S, P) = \text{IntegerPart}(B \cdot \text{Frac}(B^{n+S} \cdot \alpha_{b,c})).$$

Polynomial-time BBP splitting:

$$\alpha_{b,c} = \sum_{j=1}^{J(n)} \frac{1}{c^j b^{c^j}} + \sum_{j=J(n)+1}^{\infty} \frac{1}{c^j b^{c^j}},$$

with $J(n) = O(\log n)$.

B.4 B.4 Oracle for the Sum of Two Stoneham Constants

$$\alpha_{b,c_1} + \alpha_{b,c_2}.$$

Digit oracle:

$$f(n, c_1, c_2, S, P) = \text{IntegerPart}\left(B \cdot \text{Frac}\left(B^{n+S}(\alpha_{b,c_1} + \alpha_{b,c_2})\right)\right).$$

Polynomial-time because each summand has polynomial BBP complexity.

B.5 B.5 Polynomial Stoneham Generalization (Equation 10.10)

$$\alpha_{b,c_1} + \alpha_{b,c_2} + \cdots + \alpha_{b,c_q},$$

with b coprime to each c_j .

Digit oracle:

$$f(n, \{c_j\}, S, P) = \text{IntegerPart}\left(B \cdot \text{Frac}\left(B^{n+S} \sum_{j=1}^q \alpha_{b,c_j}\right)\right).$$

B.6 B.6 Polynomial-Time Oracle for Polynomial Stoneham Sums

$$\alpha_{b,c,P} = \sum_{k=1}^{\infty} \frac{1}{c^{P(k)} b^{c^{P(k)}}}.$$

Digit oracle:

$$f(n, c, P, S, P) = \text{IntegerPart}\left(B \cdot \text{Frac}\left(B^{n+S} \cdot \alpha_{b,c,P}\right)\right).$$

Polynomial-time because $P(k)$ grows polynomially and the BBP splitting remains polynomial.

Extension to Korobov-type constants. In addition to Stoneham-type constants, our quantum oracle construction extends to Korobov-type constants of the form

$$\beta_{w,x,z} = \sum_{n=x^z, x^{z^2}, x^{z^3}, \dots} \frac{1}{n w^n},$$

which were introduced in [1] and are known to be b -normal for bases b satisfying the usual coprimality conditions. Each term in $\beta_{w,x,z}$ has the same left-half/right-half BBP splitting structure as the Stoneham constants $\alpha_{b,c}$, since the exponents x^{z^k} grow doubly exponentially and the tail of the series converges geometrically. Consequently, the digit-extraction oracle for $\beta_{w,x,z}$ runs in polynomial time in $\log n$, exactly as for Stoneham constants. More generally, finite sums combining a *polynomial number* p of Korobov-type constants with a *polynomial number* q of Stoneham-type constants,

$$\beta_{w_1,x_1,z_1} + \beta_{w_2,x_2,z_2} + \cdots + \beta_{w_p,x_p,z_p} + \alpha_{b,c_1} + \alpha_{b,c_2} + \cdots + \alpha_{b,c_q},$$

retain a polynomial-time digit-extraction oracle, because each of the $p + q$ summands admits a polynomial-time BBP evaluation individually, and the oracle for the total sum is obtained by composing these $p + q$ BBP circuits with the same fractional-part composition identity used in B.4 and B.8 below, $\text{Frac}(B^n(g + h)) = \text{Frac}(\text{Frac}(B^n g) + \text{Frac}(B^n h))$, applied iteratively across all $p + q$ terms. Since $p + q$ is polynomial (in particular, independent of n), this composition changes the overall digit-extraction cost by only a polynomial factor, so the combined oracle remains polynomial-time in $\log n$. Thus the generalized QBNTA and amplitude-estimation algorithms developed in this paper apply equally to Korobov-type constants, Stoneham-type constants, and to any finite, polynomial-sized mixture of the two, providing a unified quantum framework for testing simple b -normality across a broad class of BBP-type series. We note that this extension inherits the same complexity caveats developed above: it broadens the class of constants to which QBNTA applies, but does not by itself change whether the fixed- K or amplitude-estimation variant is used, nor the speed-up (or absence thereof) established for each in Sections 4.3 and A.2.

B.7 B.7 Oracle for Rational Multiples of Stoneham Constants

Let $r = p/q \in \mathbb{Q}$ be in lowest terms. A subtlety must be handled correctly here: it is *not* valid to obtain the digit oracle for $r \alpha_{b,c}$ by computing $\text{frac}(B^n \alpha_{b,c})$ with the ordinary BBP splitting of Section B.3 and then multiplying the result by r . Writing $B^n \alpha_{b,c} = I + f$ with $I = \lfloor B^n \alpha_{b,c} \rfloor$ and $f = \text{frac}(B^n \alpha_{b,c})$, the quantity actually required is

$$\text{frac}(B^n r \alpha_{b,c}) = \text{frac}(rI + rf),$$

and $\text{frac}(rI)$ need not vanish when $q \nmid I$; it depends on $I \bmod q$, which the ordinary BBP algorithm never computes (avoiding the exponentially large integer I is the point of that algorithm). Hence $r \cdot f \neq \text{frac}(B^n r \alpha_{b,c})$ in general, and the oracle must instead be built by re-deriving the BBP splitting directly on the rescaled series

$$r \alpha_{b,c} = \sum_{j=1}^{\infty} \frac{p}{q c^j b^{c^j}},$$

using modulus $q c^j$ in place of c^j throughout. Since q is a fixed constant independent of n , this changes the modular exponentiation cost by only a constant factor, and the resulting digit oracle

$$f_{r,\alpha}(n, c, b) = \text{IntegerPart} \left(b \cdot \text{Frac} \left(\sum_{j=1}^{J(n)} \frac{p (b^{n-c^j} \bmod (q c^j))}{q c^j} + \sum_{j=J(n)+1}^{\infty} \frac{p b^n}{q c^j b^{c^j}} \right) \right)$$

remains polynomial-time in $\log n$, with $J(n) = O(\log n)$ exactly as in Section B.3.

Remark 7 (for single Stoneham constant). For a *single* Stoneham constant, Wall's theorem on normality already tells us that if $\alpha_{b,c}$ is b -normal (as it is, by Theorem 1 above, for every coprime pair b, c), then $r \alpha_{b,c}$ is also b -normal for every nonzero rational r : normality is preserved under multiplication by a nonzero rational (and addition of a rational constant) [2], a fact already recorded as Theorem 2.1 in the classical literature cited in our introduction. So for the single-term case treated in this subsection, running QBNTA or AE-QBNTA on $r \alpha_{b,c}$ to gather normality evidence is, strictly speaking, unnecessary: the answer is already a theorem.

The oracle constructed here nonetheless is required to be mention here because it is exactly the building block B.8 needs, and Wall's theorem does *not* extend from single terms to sums. It is a well-known phenomenon⁷ and, for Stoneham constants specifically, an open problem in the literature⁸ that normality is not preserved under addition: even though α_{b,c_1} and α_{b,c_2} are each individually b -normal, it is not known whether $\alpha_{b,c_1} + \alpha_{b,c_2}$ is b -normal, and the same is true after each summand is separately rescaled by a nonzero rational, since rescaling does not change whether addition preserves normality. This holds regardless of how many summands are combined: the finite (even polynomial-sized, in the number of terms summed) sums

treated in B.8 below are not covered by Wall’s theorem for exactly this reason, and testing them with QBNTA/AE-QBNTA is therefore gathering evidence about a genuinely open question, not re-deriving a known theorem. This is the sense in which B.7’s oracle is a necessary, non-redundant ingredient for B.8, even though it is redundant on its own.

B.8 B.8 Oracle for Rational-Linear Combinations of Stoneham Constants

The construction of B.7 extends termwise to any finite rational-linear combination

$$r_0 + r_1\alpha_{b,c_1} + r_2\alpha_{b,c_2} + \cdots + r_q\alpha_{b,c_q}, \quad r_j = p_j/q_j \in \mathbb{Q},$$

by the same fractional-part composition identity used in B.4, $\text{Frac}(B^n(g + h)) = \text{Frac}(\text{Frac}(B^n g) + \text{Frac}(B^n h))$, applied here to each rescaled summand $r_j\alpha_{b,c_j}$ (with the B.7 oracle, with its own modulus $q_j c_j^k$) together with the constant term r_0 , whose base- b expansion is itself eventually periodic and trivially computable in $O(1)$ time per digit. The combined oracle

$$f(n, \{r_j\}, \{c_j\}, b) = \text{IntegerPart}\left(b \cdot \text{Frac}\left(r_0 + \sum_{j=1}^q \text{Frac}(B^n r_j \alpha_{b,c_j})\right)\right)$$

is a composition of $q + O(1)$ polynomial-time BBP-type evaluations and is therefore itself polynomial-time in $\log n$. Consequently QBNTA and AE-QBNTA apply unchanged to any such rational-Stoneham combination, enlarging the class of testable constants while preserving the polynomial-time complexity of the oracle; this is the same conclusion reached in the original submitted paragraph, but obtained by rescaling the series before splitting rather than rescaling the fractional part after it.

C QBNTA Using Quantum Amplitude Estimation (Quadratic-Speed Version)

This appendix gives a complete, explicit, step-by-step version of QBNTA in which the central-limit-theorem sampling procedure is replaced by quantum amplitude estimation (QAE). The structure mirrors Appendix B: the algorithm is written out as numbered steps, beginning with state preparation

and ending with a null-hypothesis decision. The estimator is obtained with phase estimation rather than repeated measurement.

Algorithm 3 AE-QBNTA: QBNTA with Quantum Amplitude Estimation (Base 2)

[1]

Step 1: Inputs.

- A real constant c with a BBP-type digit-extraction formula.
- Starting position S and block length 2^a , so the block runs from S to $P = S + 2^a$.
- Digit oracle $f(n, c, S, P) \in \{0, 1\}$ returning the $(n + S)$ -th binary digit of c .
- Precision parameter $\varepsilon > 0$ determining the required accuracy of the Grover angle.
- A confidence level (e.g. the BHMT constant $8/\pi^2$).

Step 2: State preparation operator A .

- Prepare the position register in the uniform superposition

$$\frac{1}{\sqrt{2^a}} \sum_{n=0}^{2^a-1} |n\rangle.$$

- Attach a work qubit initialised to $|0\rangle$.
- Apply the digit oracle so that the work qubit flips to $|1\rangle$ exactly when $f(n, c, S, P) = 1$. when computing this digit oracle in superposition over an exponential number of values
- This defines the “good” and “bad” subspaces.

Step 3: Define the Grover operator Q .

- Construct the Grover operator

$$Q = -AS_0A^{-1}S_f,$$

where S_0 is the reflection about $|0\rangle$ and S_f is the reflection about the marked subspace.

- The initial state decomposes into good/bad components with amplitudes $\sin \theta$ and $\cos \theta$, where

$$\theta = \arcsin\left(\sqrt{\frac{M}{2^a}}\right),$$

and M is the number of ones in the block.

Step 4: Prepare the ancilla register.

- Choose an integer t such that $2^{-t} \leq \varepsilon$.
- Prepare an ancilla register of t qubits in equal superposition:

$$\frac{1}{2^{t/2}} \sum_{y=0}^{2^t-1} |y\rangle.$$

- This register stores the phase information.

Step 5: Apply controlled powers of Q .

- For each ancilla qubit $j = 0, \dots, t-1$:
 - If the ancilla qubit is $|1\rangle$, apply Q^{2^j} to the main register.
 - If it is $|0\rangle$, do nothing.
- This encodes the eigenphase $\pm 2\theta$ into the ancilla register in the standard QAE pattern.

Step 6: Apply the inverse quantum Fourier transform.

- Apply the inverse QFT to the ancilla register.
- This converts the phase information into a binary integer.

Step 7: Measure the ancilla register.

- Measure the ancilla register to obtain an integer $y \in \{0, \dots, 2^t - 1\}$.
- Recover the estimate of the Grover angle:

$$\hat{\theta} = \frac{\pi y}{2^t}.$$

- The BHMT bound guarantees

$$|\hat{\theta} - \theta| \leq \frac{\pi}{2^t}$$

with constant success probability.

Step 8: Null-hypothesis decision.

- Under simple normality, the proportion of ones is $1/2$, so $\theta = \pi/4$.
- This corresponds to the ancilla outcome $y_0 = 2^{t-2}$.
- For a fixed-confidence binary decision:
 - Check the leading bit of the measured y .
 - If it disagrees with the leading bit of y_0 , reject H_0 .
 - If it agrees, fail to reject H_0 .
- Only the leading bit is needed because QAE already provides precision ε .

Step 9: Output.

- The decision “balanced” or “biased”.
- The measured integer y .
- The angle estimate $\hat{\theta}$.
- Optionally, classification into the same five qualitative outcomes as Appendix B: exact balance, all ones, all zeros, small bias, or erratic/Weyl-distributed behaviour.

Quadratic speed-up. The controlled powers of Q require 2^j oracle calls for the j -th ancilla qubit. Thus one run of the QAE circuit uses

$$O(2^t) = O(1/\varepsilon)$$

oracle calls. By contrast, the CLT-based estimator in Appendix B requires

$$O(1/\varepsilon^2)$$

oracle calls to resolve a bias of size ε . Replacing repeated measurement with amplitude estimation therefore yields a quadratic speed-up, and this speed-up is provably optimal in the BHMT query model.

D A Classical Digit-Extraction Formula for the Euler–Mascheroni Constant, a Quantum Oracle for the Euler–Mascheroni Constant, and Quantum Algorithms for testing Euler–Mascheroni Constant is Rational

D.1 D.1 Introduction

In this appendix D we show that a known truncated asymptotic series for the Euler–Mascheroni constant γ can be evaluated, at a sparse family of positions $d \approx 2^{2^g}$ ($g \in \mathbb{Z}^+$), by a direct closed-form computation rather than by the iterative precision-refinement used in standard high-precision algorithms (e.g. Brent–McMillan). Because the series is hypergeometric-type (its term ratio is a rational function of the summation index), binary splitting applies, giving time complexity $O(d(\log d)^{O(1)})$ and space complexity $O(d \log d)$; both of the sub-exponential form $O(d^Q(\log d)^W)$ for fixed constants Q, W , and in particular not exponential in d . We are explicit about what this result does and does not claim: it is not a digit-extraction algorithm in the Bailey–Borwein–Plouffe (BBP) sense (it uses arbitrary-precision arithmetic, not bounded modular reduction) but it is a digit extraction algorithm in the sense that to compute the target binary digit for the Euler–Mascheroni constant γ at sparse positions 2^{2^g} the previous digits up to that position need not be computed; it is not shown to be faster than existing quasi-linear algorithms for γ , but it is faster than exponential time $O(2^d)$, which makes this classical (i.e. non-quantum) digit-extraction algorithm for the Euler–Mascheroni constant γ worth recording; and it applies only at this sparse, double-exponentially spaced set of positions rather than at an arbitrary digit d .

High-precision computation of γ is normally done with iterative algorithms; most notably Brent–McMillan, which uses a doubling (AGM-style) precision-refinement process and runs in $O(d \log^2 d)$ time with fast multiplication. Such algorithms are *iterative*: the computation at working precision d is built up through a sequence of refinements, and no single finite closed-form expression hands you the answer directly.

The present appendix starts from a different kind of object: a known truncated asymptotic expansion for γ in terms of an integer parameter k ,

with an explicit, boundable error term. (We take this base series as given and numerically verified; see Section D.2.) Such a series is a single finite sum, not a refinement loop, and the question we address is: how efficiently can that finite sum itself be evaluated?

What we show. For the special choice $k = 2^{2^g}$, the resulting sum has $\Theta(k)$ terms whose consecutive ratio is a rational function of the summation index. This is precisely the structural condition under which *binary splitting* (Chudnovsky–Haible–Papanikolaou) applies, and it yields a bit-complexity of $O(d(\log d)^{O(1)})$ time and $O(d \log d)$ space for extracting $d \approx k$ correct bits of γ – sub-exponential in d , and obtained without running any iterative refinement over lower precisions. This is a new structural observation: it is a different *route* to bits of γ at these special positions, not a faster one, and not a BBP-style extraction algorithm, but an alternative, still valid, digit-extraction algorithm for the Euler–Mascheroni constant γ .

What this appendix does not claim.

- It does not claim to beat, or match the practical performance of, Brent–McMillan or other quasi-linear algorithms for γ .
- It does not produce a BBP-type extraction algorithm: BBP algorithms compute digit d using $O(\log d)$ -bit modular arithmetic and never form a d -bit object; this method uses arbitrary-precision arithmetic on numbers with $\Theta(d \log d)$ bits internally (reduced to $O(d \log d)$ space via binary splitting, not $O(\log d)$). BBP also uses arbitrary-precision arithmetic, but with integers only, for the bulk of its computation.
- It applies only at the sparse set of positions $d \approx 2^{2^g}$ for integer g , not at an arbitrary target digit d – though Section D.9 below shows how to reach an arbitrary d regardless, at a cost that remains polynomial in d .

D.2 D.2 The Base Series

We take as given the following truncated expansion for γ , given in [6], in terms of a positive integer parameter k :

$$\gamma = 1 - \log(k) \sum_{r=1}^{12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2} + O(2^{-k}). \quad (1)$$

Write $A(k) = \sum_{r=1}^{12k+1} a_r$ with $a_r = (-1)^{r+1} k^{r+1} / [(r-1)!(r+1)]$, and $B(k) = \sum_{r=1}^{12k+1} b_r$ with $b_r = (-1)^{r+1} k^{r+1} / [(r-1)!(r+1)^2]$, so that $\gamma = 1 - \log(k)A(k) + B(k) + O(2^{-k})$. The value $\log k$ is itself computed to the needed number of bits via a standard BBP-type series for $\log 2$ (used only because k is chosen to be a power of two below; $\log k$ then reduces to an integer multiple of $\log 2$, computed separately and to sufficient precision).

D.3 D.3 Reachable Positions

The error term $O(2^{-k})$ in (1) means the truncated expression yields roughly k correct bits of γ , i.e. the reachable bit-position is $d = \Theta(k)$. We choose

$$k = 2^{2^g}, \quad g = 1, 2, 3, \dots$$

This choice is not necessary for the series itself; it is a convenience that makes k a power of two, which simplifies the fixed-point/binary algorithm needed to isolate the fractional part $\{2^d \gamma\}$ and extract a specific bit. The consequence is that this method, as described here, only targets the sparse, double-exponentially spaced family of positions $d \approx 2^{2^g}$; not an arbitrary integer d directly (Section D.9 removes this restriction, at a cost). (Reaching intermediate positions *exactly* at the same k would require re-deriving the error bound for non-power-of-two k , which we have not done; Section D.9 instead simply rounds up to the next reachable k .)

The precise relationship between g and the number of guaranteed-correct bits, together with the auxiliary precision h needed for $\log k$, is carried over from the error analysis in Section D.8. In particular g must grow with the target position (roughly $g = \Theta(\log \log d)$); it cannot be held at a small fixed constant while $d \rightarrow \infty$, as an earlier draft of this note incorrectly implied.

D.4 D.4 Why Binary Splitting Applies

Binary splitting computes a sum $S = \sum_{r=0}^N p_r/q_r$ efficiently when the summand can be written as a product telescoping via integer sequences p_r, q_r satisfying a rational recurrence, by recursively splitting the range of summation and combining partial numerators and denominators; without ever forming the full-precision decimal value until the final combination step. The standard applicability condition is that the ratio of consecutive terms be a rational function of the index.

For our series, with $a_r = (-1)^{r+1}k^{r+1}/[(r-1)!(r+1)]$,

$$\frac{a_{r+1}}{a_r} = \frac{-k^{r+2}/[r!(r+2)]}{k^{r+1}/[(r-1)!(r+1)]} = \frac{-k(r+1)}{r(r+2)}, \quad (2)$$

which is a rational function of r (numerator and denominator each of bounded degree, with k a fixed integer parameter for the duration of the sum). The same holds for b_r , with an extra factor of $(r+1)/(r+2)$ arising from the extra $(r+1)$ in the denominator. This is exactly the hypergeometric-term condition under which binary splitting is applicable; both $A(k)$ and $B(k)$ can therefore each be evaluated by a single binary-splitting recursion, and the final combination $1 - \log(k)A(k) + B(k)$ is one further multiplication and two additions.

D.5 D.5 Complexity

Let d denote the target bit-position, so $k = \Theta(d)$ and the number of summation terms is $N = 12k + 1 = \Theta(d)$. The integer k itself has bit-length $L = \log_2 k = \Theta(\log d)$. The largest terms in the sum (near $r \approx 12k$) have numerator k^{r+1} of bit-length $\Theta(r \cdot L) = \Theta(d \log d)$, and similarly-sized factorial denominators; so naively expanding every term would require handling numbers with $\Theta(d \log d)$ bits, $\Theta(d)$ times over; an $\Omega(d^2 \text{polylog } d)$ computation if done term-by-term with no shared structure.

Binary splitting avoids this by combining terms pairwise in a balanced recursion tree of depth $O(\log d)$, so that the bit-length of the numerator/denominator pairs carried at recursion level i grows geometrically with i , reaching the full $\Theta(d \log d)$ bits only at the root. Using this recursion with fast multiplication $M(n) = O(n \log n)$, the standard bound for such hypergeometric sums gives total bit complexity

$$T(d) = O(M(d \log d) \log d) = O(d (\log d)^{O(1)}), \quad (3)$$

and space

$$S(d) = O(d \log d), \quad (4)$$

since the largest object ever held in memory is the root-level numerator/denominator pair, of $\Theta(d \log d)$ bits. Both bounds are of the form $O(d^Q (\log d)^W)$ for fixed constants $Q = 1$ and some fixed W ; sub-exponential in d , in particular not

$O(2^d)$, and both are achieved without any iterative refinement over intermediate precisions: the recursion operates once, on the fixed finite sum defined by the chosen k .

We repeat again what this does and does not mean. It does not mean this method is faster than Brent–McMillan (which already achieves $O(d \log^2 d)$ by a different, iterative route, and which applies at every d , not just a sparse subsequence). What it means is that a *closed-form, non-iterative* route (i.e. a digit-extraction algorithm) to d correct bits of γ exists at these special positions, with complexity of the same general shape as the best iterative methods, obtained by recognizing the hypergeometric structure of the truncated series rather than by any digit-recurrence.

D.6 D.6 Remark on a Link to Dynamical Systems

The fractional-part bookkeeping used to isolate the target bit from $\{2^d \gamma\}$ can be written as an iterated map

$$x_n = \text{Frac} \left(2x_{n-1} + \text{Frac} \left(\frac{2^g}{n} \right) \right). \quad (5)$$

We then ask whether equidistribution properties of this map say anything about the normality of γ . We flag this only as a speculative analogy: we have not established ergodicity, equidistribution, or any rigorous link between this iterated map and the digit statistics of γ , and no such claim should be read into this section. It is included as a possible direction for future work, not as a result. (We do not draw on this speculative remark anywhere in Section D.9 below; the quantum oracle constructed there uses only the base series of Section D.2 and the error bounds of Section D.8, not this dynamical analogy.)

D.7 D.7 Conclusion of the Classical Construction

A known truncated series for γ , evaluated at the sparse double-exponential family of parameters $k = 2^{2^g}$, can be computed by binary splitting rather than term-by-term summation. This gives a direct, non-iterative route to $d \approx k$ correct bits of γ with time complexity $O(d(\log d)^{O(1)})$ and space complexity $O(d \log d)$; both comfortably sub-exponential in d , though restricted to this sparse position family and not shown to be competitive with existing

quasi-linear iterative algorithms. We regard the main contribution as structural: identifying that this class of asymptotic series for γ is hypergeometric and therefore binary-splittable, giving a concrete, checkable alternative to iterative computation at these positions.

D.8 D.8 Error Bounds and Sufficient Term Derivation

To guarantee correct extraction of the target bit, the error introduced by truncating the BBP-type series used for $\log k$ must be bounded well below the target precision. Let h be the number of bits computed for $\log k$; we take

$$h = 13g \cdot 2^g + 13g + 10,$$

so the truncation error in $\log k$ is $\epsilon \leq 2^{-h}$. This propagates into the finite sum as

$$\sum_{r=1}^{12k+1} \left(\epsilon \frac{k^{r+1}}{(r-1)!(r+1)} \right) \pmod{1}. \quad (6)$$

Substituting $k = 2^g$ momentarily to bound the largest term (at $r = 12(2^g) + 1$), the numerator is $(2^g)^{12(2^g)+2}$; combined with $\epsilon = 2^{-h}$, the resulting fractional contribution from the largest term is bounded by $2^{-g^{2^g}}$, and summing the (at most $12 \cdot 2^g + 1$) terms gives a total error bounded above by

$$\frac{12 \cdot 2^g + 1}{2^{g^{2^g}}},$$

which is strictly less than $1/2$ for all $g \geq 3$, confirming that h bits of precision in $\log k$ suffice to avoid corrupting the target bit.

Separately, the intrinsic error of the base series itself, $O(2^{-k})$ (more precisely on the order of $2k^k e^{-k}$), must also fall below the target precision. Since $k = 2^{2^g}$, this requires the target bit-position d to satisfy $d \lesssim k$; equivalently, g must grow with d (specifically $g = \Theta(\log \log d)$) rather than being held at a fixed constant. Under this scaling, both error sources are controlled simultaneously, and the construction yields $d \approx k$ correct bits for every $g \geq 3$.

D.9 D.9 A Quantum Oracle for Testing Binary Simple Normality of γ

We now derive a digit-extraction oracle for γ directly from the base series of Section D.2, and use it to give a quantum algorithm (QBNTA or AE-QBNTA,

exactly as defined in Section 4.3 and Appendix A, with no changes to those constructions) for gathering statistical evidence about γ 's digit behaviour at the sparse family of positions the base series naturally reaches. We show, however, that this oracle is not polynomial-time in the sense needed elsewhere in this paper, so the resulting algorithm is not an efficient one – a fact worth recording precisely rather than glossing over, since γ is exactly the kind of constant (like π) whose normality is of genuine interest despite the lack of an efficient oracle.

The oracle, as a single closed-form expression. Let the position register hold a qubits representing $m \in \{0, \dots, 2^a - 1\}$, and set $g(m) = m + 3$ (so $g \geq 3$ throughout, as Section D.8 requires). The oracle tests the single bit at position

$$n(m) = k(m) = 2^{2^{g(m)}} = 2^{2^{m+3}},$$

by directly evaluating the base series at $k = k(m)$ and reading off bit $n(m)$. Since 1 is an integer and $2^n \cdot 1$ is even for every $n \geq 1$, and $\lfloor x + y \rfloor = x + \lfloor y \rfloor$ for integer x , the leading “1–” in (1) contributes $2^{n(m)} \bmod 2 = 0$ to the extracted bit and can simply be dropped:

$$R(m) = \lfloor 2^{n(m)} (B(k(m)) - \log(k(m)) A(k(m))) \rfloor \bmod 2, \quad (7)$$

with $A(k), B(k)$ the finite sums of Section D.2, and $\log(k(m)) = 2^{g(m)} \log 2$ computed to $h(g(m))$ bits via the BBP-type series for $\log 2$ (Section D.8); $h(g(m))$ guard bits are computed so the BBP truncation error is below the target precision. It is $g(m)$, more precisely, the value $2^{g(m)} \log 2$ that gets substituted into the $\log 2$ BBP series, that must be substituted to make evaluation fall within the time and space bounds of Section D.5, which is why the oracle is built directly around $g(m)$ (equivalently m) rather than around an arbitrary target bit position: the base series of Section D.2 is only ever evaluated at $k = 2^{2^g}$, so g (placed in the position register) is the natural quantity to superpose over, and the oracle only reaches positions of the form $2^{2^{(\cdot)}}$.

Because the parameter choice $k = 2^{2^g}$ implies $\log k = 2^g \log 2$, the logarithmic term reduces to evaluating $2^g \log 2 \pmod{1}$. This allows the use of the standard BBP series for $\log 2$, which computes the required fractional part using modular exponentiation and arbitrary-precision *integer* arithmetic only, avoiding the need for arbitrary-precision floating-point computation.

Because the algorithm combines a finite hypergeometric series with a BBP-evaluated logarithmic term ($2^g \log 2$), the total numerical error is the sum of the truncation error from (1) and the BBP truncation error. Both components must therefore be evaluated with sufficient precision, including guard bits beyond the target bit accuracy, to ensure that the combined error remains below the target precision and does not affect the extracted digit.

Equation (7) is deliberately written as one closed-form expression, evaluated by direct term-by-term summation of $A(k(m))$ and $B(k(m))$; it does not invoke the binary-splitting recursion of Section D.4 at all. Binary splitting in Section D.4 is an *implementation* that evaluates $A(k)$ and $B(k)$ faster; the oracle itself is the formula (7), and can equally well be evaluated (more simply, if less efficiently) by directly summing the $12k(m) + 1$ terms of each sum one at a time.

How the log-term is actually evaluated: one BBP call, not one per term. It is worth stating precisely how (7) is evaluated, since the mechanism is what keeps the complexity bound of Section D.5 intact rather than inflating it. Write $N = n(m) + g(m)$. Because $\log(k(m)) = 2^{g(m)} \log 2$, the quantity $2^{n(m)} \log(k(m)) = 2^N \log 2$ is exactly what a standard BBP evaluation of $\log 2 = \sum_{j \geq 1} 1/(j2^j)$ computes at shift N : splitting at a truncation point s ,

$$2^N \log 2 \bmod 1 = \underbrace{\left(\sum_{j=1}^{s-1} \frac{2^N}{j2^j} \right) \bmod 1}_{=:R} + \underbrace{\left(\sum_{j=s}^{\infty} \frac{2^N}{j2^j} \right) \bmod 1}_{=: \varepsilon}.$$

Crucially, R is computed *once*, a single BBP-type evaluation, and does not depend on which term of $A(k(m))$ it will later multiply. Since $A(k(m)) = \sum_{r=1}^{12k(m)+1} a_r$,

$$2^{n(m)} \log(k(m)) A(k(m)) = (R + \varepsilon) A(k(m)) = R \cdot A(k(m)) + \varepsilon \cdot A(k(m)),$$

and the first term is $R \cdot A(k(m)) = \sum_r (R \cdot a_r)$: the same finite hypergeometric sum as $A(k(m))$ itself, with every term scaled by the single fixed rational R . Scaling a hypergeometric sum's terms by a constant does not change the rational-term-ratio property of Section D.4, so this is evaluated by one application of binary splitting to $A(k(m))$ followed by one multiplication by R ; not by $12k(m) + 1$ separate BBP evaluations, one per term. This is the point that keeps the log-term's contribution inside the same $O(k(m)(\log k(m))^{O(1)})$ bound as $A(k(m))$ itself, rather than a further factor of $k(m)$ worse.

The remaining term here, $\varepsilon \cdot A(k(m))$, is exactly the error term bounded in Section D.8: with $\varepsilon \leq 2^{-h}$ for $h = 13g(m)2^{g(m)} + 13g(m) + 10$, the term-by-term bound derived there gives $\sum_r |\varepsilon \cdot a_r| < 1/2$ for all $g(m) \geq 3$, so this error cannot flip the extracted bit. Computing R to this precision costs $O(N)$ terms for the left half of the $\log 2$ BBP sum (each reduced via fast modular exponentiation) plus $O(h)$ further terms for the geometric tail ; and it is worth being explicit that this tail needs $\Theta(h)$ terms, not $O(\log h)$: each additional term buys only a constant number of extra bits of precision, a standard feature of BBP-type tails that is easy to misread as logarithmic if one is not careful. Both term counts, $O(N)$ and $O(h) = O(g(m)2^{g(m)})$, are dominated by the $A(k(m)), B(k(m))$ cost of Section D.5 (since $g(m)2^{g(m)}$ is negligible next to $k(m) \log k(m)$), so computing R does not change the overall order of the complexity bound. The claim that a single evaluation runs in $O(k(m)^Q (\log k(m))^W)$ time and comparable space for this half of the series, for fixed constants Q, W , holds exactly as stated in Section D.5 ; as a function of $k(m)$, though not, as the next paragraph shows, as a function of the register size a .

The other half of the series: $B(k)$ needs no log-BBP Type Analysis, and the same bound already covers it. Unlike $A(k)$, which is multiplied by $\log k$ and hence needs the BBP-based treatment above, $B(k)$ enters (1) directly, so extracting its contribution to bit $n(m)$ only requires $2^{n(m)} B(k(m)) \bmod 1 = \sum_r 2^{n(m)} b_r \bmod 1$ – no logarithm, and hence no separate BBP evaluation, is involved at all. Here $b_r = (-1)^{r+1} k^{r+1} / [(r-1)!(r+1)^2]$, with $(r+1)^2$ an ordinary integer squared; the only factorial present is the $(r-1)!$ in the denominator, exactly as in a_r .

The source establishing the base identity (1) gives the following sharper, two-sided form of its $O(2^{-k})$ error term [6]:

$$\begin{aligned} - \left(\frac{2}{(12k)!} + 2k^2 e^{-k} \right) &\leq \gamma - 1 + \log k \sum_{r=1}^{12k+1} \frac{(-1)^{r-1} k^{r+1}}{(r-1)!(r+1)} - \sum_{r=1}^{12k+1} \frac{(-1)^{r-1} k^{r+1}}{(r-1)!(r+1)^2} \\ &\leq \frac{2}{(12k)!} + 2k^2 e^{-k}, \quad k \geq 1. \end{aligned}$$

Two things follow from this. First, since $(12k)!$ grows super-exponentially and e^{-k} decays faster than 2^{-k} (as $e > 2$), the combined error $\frac{2}{(12k)!} + 2k^2 e^{-k}$ is $o(2^{-k})$: truncating *both* $A(k)$ and $B(k)$ at exactly $12k + 1$ terms already

gives an approximation far more accurate than the target $O(2^{-k})$ precision quoted in (1), with room to spare. No further terms, beyond the $12k + 1$ already used, are ever needed for either sum.

Second, because $B(k)$ is a hypergeometric sum of exactly the same kind as $A(k)$ (Section D.4 already establishes the rational term-ratio for b_r directly), the same binary-splitting treatment of Sections D.4–D.5 applies to it without modification, at the same $O(k(\log k)^{O(1)})$ cost ; no separate term count analysis is needed beyond what is already given there, since $B(k)$'s $12k + 1$ terms were always part of that accounting. There is also a second, way to see the same thing, closer to how one would implement this directly rather than via binary splitting: because $(r - 1)!$ in the denominator grows so much faster than $k^{r+1} \cdot 2^{n(m)}$ in the numerator once r is a modest multiple of k (factorial growth outpaces any fixed exponential rate, including the $2^{n(m)}$ scaling used for bit extraction), and because $k = 2^{g(m)}$ is itself a power of two, the factors of 2 in $2^{n(m)}$ and in k^{r+1} can be cancelled directly against the factors of 2 already present in $(r - 1)!$; in the same spirit as the modular-reduction trick used in the standard BBP algorithm ' rather than requiring every term to be formed as one unreduced, full-precision integer before combining. Either route keeps $B(k(m))$'s contribution inside the same $O(k(m)^Q(\log k(m))^W)$ bound as $A(k(m))$.

Consequence: doubly and triply exponential in the quantum register size. As m ranges over the full a -qubit register, $g(m) = m + 3$ ranges up to $2^a + 2$: g is tied *linearly* to the register value, so $g(m) = \Theta(2^a)$ at the top of the register. Consequently

$$\log_2 k(m) = 2^{g(m)} = 2^{\Theta(2^a)} :$$

doubly exponential in a . And $k(m)$ itself is $2^{\log_2 k(m)} = 2^{2^{\Theta(2^a)}}$: *triply* exponential in a ; a tower of the form $2^{2^{2^a}}$, not 2^{2^a} . Since the intermediate bit-length $\Theta(k(m) \log k(m))$ of Section D.5 is dominated by $k(m)$ itself (the extra factor $\log k(m)$, though doubly exponential, is negligible next to the triply-exponential $k(m)$), the qubit count needed for a single oracle call ; and its running time ; are both *triply* exponential in a .

Concretely: with only $a = 5$ qubits, m already reaches 31, giving $g(m) = 34$ and $\log_2 k(m) = 2^{34} \approx 1.7 \times 10^{10}$;so $k(m)$ itself is a number with about seventeen billion *bits* (roughly five billion decimal digits). For comparison, the number of atoms in the observable universe is about $10^{80} \approx 2^{266}$, a

number needing only 267 bits; $k(m)$ at $a = 5$ already needs some 1.7×10^{10} bits just to write down, let alone the further $k(m) \log k(m)$ -bit registers the arithmetic itself would need.

This is a stronger statement than "exponential time": it says the number of *qubits* needed for the intermediate arithmetic, not only restricted to the number of gates applied to them, grows triply exponentially with the size of the position register. The construction is therefore not practically instantiable as a circuit at all beyond a handful of qubits. The number of oracle *calls* QBNTA/AE-QBNTA needs is, as always, unaffected by any of this, still $O(a)$ or $O(1/\varepsilon)$, exactly as established in Section 4.3 and Appendix A, but here it is the size of a single call's circuit, not just its running time, that is unworkable.

Speculative remark: what a single fixed- g evaluation might reach under hypothetical future qubit counts. The triply-exponential growth above is about the full g -in-superposition construction, where g ranges over an entire a -qubit register. It is worth separately asking, purely as an illustrative, speculative exercise and not as a claim about this paper's algorithm, what a single *fixed* choice of g – one classical (or Bennett-reversible) evaluation of the oracle at one target position $n = k = 2^{2^g}$, with no register held in superposition at all; would require, and whether that could ever be within reach of hypothetical future hardware. By Section D.5, such an evaluation needs working registers of size $\Theta(k \log_2 k)$ bits. For $g = 4$, $k = 2^{16} = 65536$ and $k \log_2 k \approx 1.05 \times 10^6$: on the order of one million qubits, reaching digit position $\approx 2^{16}$. For $g = 5$, $k = 2^{32} \approx 4.3 \times 10^9$ and $k \log_2 k \approx 1.4 \times 10^{11}$: on the order of 140 billion qubits, reaching digit position $\approx 2^{32}$ – roughly 137,000 times more qubits than $g = 4$ needed, for one further step in g . If hardware roadmaps of the kind sometimes discussed in the field were to materialize; for instance, quantum computers reaching on the order of a million qubits by mid-century and perhaps a billion by the century's end, extrapolations we neither endorse nor have any basis to assess here; a million qubits would put $g = 4$ (digit position in the tens of thousands) at the edge of plausibility, while a billion qubits would still fall roughly two orders of magnitude short of $g = 5$ (digit position $\approx 2^{32}$), which needs closer to 10^{11} qubits by this count. This is offered only to make the scale of the triply-exponential wall concrete, not as a technology forecast or as evidence that this paper's oracle is practical at any foreseeable qubit count.

A caveat on what this construction actually tests. Independent of complexity, it is worth being precise about what statistical claim this construction gathers evidence for. Testing the oracle at $n(m) = 2^{2^g(m)}$ for $m = 0, \dots, 2^a - 1$ tests the digit frequency along the specific sparse subsequence $\{2^{2^g}\}_{g \geq 3}$ of bit positions ; not simple normality of γ in the ordinary sense used throughout the rest of this paper, which concerns digit frequencies within long *contiguous* blocks. These are logically different properties: simple normality of γ neither implies nor is implied by a limiting frequency of $1/2$ along this particular sparse, doubly-exponentially-spaced subsequence of positions. Everything QBNTA or AE-QBNTA reports here should be read as evidence about this sparse subsequence specifically, not as evidence about γ 's simple normality in the sense used elsewhere in this paper.

What this construction does give. Equation (7), together with QBNTA or AE-QBNTA exactly as already defined, is a completely well-specified quantum procedure built from a single closed-form expression on the base series of Section D.2. It is not an efficient algorithm, for the reasons given above, but it is a conceptually interesting one: a correct (if very expensive) quantum protocol for gathering statistical evidence about γ 's digit behaviour at this sparse family of positions, stated precisely enough to be clear about exactly where and why it fails to be efficient, and exactly what statistical claim it actually tests. Still it gives a new quantum way to look at the old problem of proving if γ is rational or irrational.

Two different empirical signals this construction can surface. Two distinct, complementary observations follow from having this oracle, corresponding to two different statistical questions and, correspondingly, two different tools already in this paper.

A frequency-bias signal, via QBNTA/AE-QBNTA directly. This is the most direct use of the oracle: run QBNTA or AE-QBNTA on it exactly as intended, and read off one of the five outcomes of Section 5. If the digits extracted at these sparse positions come back strongly and persistently biased , nearly all ones, nearly all zeros, or a small but statistically significant and stable bias, rather than the $\hat{p}_K \approx 1/2$ expected under H_0 , this is empirical evidence, in exactly the same sense as a CLT hypothesis test elsewhere in this paper, that something unusual is happening at these particular positions. The logic is the same as any such test: by analogy if a fair coin is flipped thirty

times and comes up heads every time, that outcome is not impossible under the fair-coin hypothesis, but it is so unlikely that a reasonable observer takes it as evidence against the coin being fair. Likewise, a strong and persistent bias at the sparse positions $\{2^{2^g}\}$ would be evidence against γ behaving generically there ; consistent with, though not proof of, γ failing to be simply normal, or with γ being rational (a periodic tail can easily produce a biased-looking sparse sample if the period happens to interact with the doubly-exponential spacing used here). This is not rigorous evidence either way ; like every other statistical outcome reported by QBNTA/AE-QBNTA in this paper ; but it uses the oracle exactly as QBNTA/AE-QBNTA are designed to be used, with no further machinery needed.

A periodicity signal, via QRNTA instead. A different, more structural observation follows if γ is assumed rational. If γ 's binary expansion is eventually periodic with period e , the sparse sampling positions $n(m) = 2^{2^{m+3}}$ satisfy the exact recurrence

$$n(m+1) = 2^{2^{m+4}} = \left(2^{2^{m+3}}\right)^2 = n(m)^2,$$

so the sequence of residues $n(m) \bmod e$ is precisely the orbit of the squaring map $x \mapsto x^2 \bmod e$ on the finite set $\{0, \dots, e-1\}$, started at $n(0) \bmod e$. Any orbit of a deterministic map on a finite set is eventually periodic , some value must repeat, by pigeonhole, after which the orbit repeats identically , so the residues $n(m) \bmod e$, and hence the sampled digit values $R(m)$ (once $n(m)$ is past γ 's pre-periodic prefix, if any), are eventually periodic in m . In special cases this degenerates to an eventually *constant* sequence ; for instance, if e is a power of two, $n(m) \bmod e$ is eventually 0 for all sufficiently large m , since $n(m)$ is itself a power of two that eventually exceeds e – but eventual periodicity, not necessarily an eventually constant value, is the guarantee for general e . A detected departure from periodicity in this sparse subsequence would therefore be evidence, in the same numerical-evidence sense as the rest of this paper, of γ 's irrationality.

QBNTA and AE-QBNTA are not the right tool to detect this second signal: they are frequency estimators (the proportion of 1s versus 0s), not period-finders, and periodicity of a sequence is not a claim about digit frequency. The tool already built for exactly this task, elsewhere in this paper, is QRNTA , the QFT-based period-finding algorithm of Section 6 , applied to the sparse subsequence $m \mapsto R(m)$ rather than to the raw digit sequence. In fact QRNTA does not even need this sparse-subsequence detour: given

this same oracle, QRNTA as already defined in Section 6 tests periodicity of γ 's raw digit sequence directly, and so is already, more directly, a quantum algorithm for gathering evidence about γ 's rationality once handed this appendix's oracle, without needing the sparse-position argument above at all. We record the sparse-subsequence observation because it is a clean, self-contained fact worth stating, not because it is the only or the best way to reach this particular question with the tools already in this paper.

We record this appendix's construction and both of these observations for the same reason we recorded the analogous π discussion in Section 3: γ 's simple normality in base 2 is, like π 's, a genuinely open question, and a correct (if very expensive, and differently-scoped) quantum protocol for probing it is worth stating precisely, including being precise about exactly where and why it fails to be efficient, exactly what it does and does not test, and which of this paper's own algorithms, QBNTA/AE-QBNTA for frequency, QRNTA for periodicity, is the right one for a given question.

Appendix E: A Generalization of the BBP Digit Extraction Algorithm

The standard Bailey–Borwein–Plouffe (BBP) algorithm uses series of the form $c = \sum_{u=0}^{\infty} \frac{p(u)}{q(u)b^u}$ to compute individual base b digits of a constant c . Previously, it was not known how to use a base b formula (where b is a power of 2) to compute base 10 digits without evaluating all preceding binary digits and performing a full base conversion. The following algorithm solves this, enabling direct base 10 extraction (useful for computing decimal digits of π , $\ln 2$, or Stoneham constants).

The Decimal Extraction Mechanism

To compute the α -th decimal digit of c using a base 2 BBP series, we rely on the binomial expansion of 10^α :

$$10^\alpha = (8 + 2)^\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} 8^{\alpha-j} 2^j = \sum_{j=0}^{\alpha} \binom{\alpha}{j} 2^{3\alpha-2j} \quad (8)$$

The decimal digits starting at position α are found by isolating the fractional part:

$$10^\alpha c \pmod{1} = \left(\sum_{j=0}^{\alpha} \binom{\alpha}{j} 2^{3\alpha-2j} c \right) \pmod{1} \quad (9)$$

This transforms the problem into computing a series of binary fractional parts at shifted positions (specifically the 3α , $3\alpha - 2$, up to $3\alpha - 2j$ binary digits), which can be efficiently extracted using the standard base 2 BBP algorithm.

Numerical Example: The 4th Decimal Digit of $\ln 2$

Consider the binary BBP series for the natural logarithm of 2:

$$\ln 2 = \sum_{j=1}^{\infty} \frac{1}{j2^j} \quad (10)$$

To find the 4th decimal digit ($\alpha = 3$), we evaluate:

$$10^3 \ln 2 \pmod{1} = (2^9 \ln 2 + 3(2^7 \ln 2) + 3(2^5 \ln 2) + 2^3 \ln 2) \pmod{1} \quad (11)$$

Using the binary BBP extraction scheme, we compute the required fractional components:

- $2^9 \ln 2 \pmod{1} \approx 0.89135644669$
- $3(2^7 \ln 2) \pmod{1} \approx 0.16851733501$
- $3(2^5 \ln 2) \pmod{1} \approx 0.54212933375$
- $2^3 \ln 2 \pmod{1} \approx 0.54517744447$

Summing these terms modulo 1 yields 0.14718055994, correctly revealing that the decimal sequence starting at the 4th position is 1471... (confirming the 4th digit is 1).

Arbitrary Base Expansion and Complexity

To extract digits in an arbitrary base M , we decompose M into its binary expansion ($M = x_1 + x_2 + \dots + x_m$, where x_i are powers of 2) and apply the multinomial theorem or we would use the multinomial expansion

$$(x_1 + x_2 + \dots + x_m)^n = \sum_{\substack{k_1+k_2+\dots+k_m=n \\ k_1, k_2, \dots, k_m \geq 0}} \binom{n}{k_1, k_2, \dots, k_m} (x_1^{k_1} x_2^{k_2} \dots x_m^{k_m}) \quad (289)$$

$$\binom{n}{k_1, k_2, \dots, k_m} = n! / (k_1! k_2! \dots k_m!)$$

:to

$$M^\alpha = (x_1 + x_2 + \dots + x_m)^\alpha \quad (12)$$

For example, base 13 is decomposed as $13 = 8 + 4 + 1$ ($m = 3$).

Applying this generalization to an existing BBP algorithm increases the original time complexity by a factor of $O(\alpha)$ and introduces a space complexity of $O(\alpha)$; the added time complexity follows from the binomial expansion. Consequently, an algorithm that originally operated in $O(\alpha)$ time will have a worst-case time complexity of $O(\alpha^2)$ when adapted for arbitrary base extraction. While loop optimisations can mitigate some overhead, this $O(\alpha)$ multiplier is a critical consideration when evaluating the practical performance benchmarks for decimal extraction of values like the Stoneham constants.